

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

Cricket Score Predictor

A Web-Based Platform for Predicting Cricket Match Scores Using Historical Data and Machine Learning

Document Version	1.0
Document Status	Final – For Faculty Evaluation
Prepared For	GLA University
Course / Program	B.Tech – Semester 3
Prepared By	Deepanshu Thakur (22515000042)
Project Guide	Dr. Gautam Mukherjee
Date	18/08/2026

Table of Contents

1. Introduction.....	3
1.1 Purpose.....	3
1.2 Document Conventions.....	3
1.3 Intended Audience and Reading Suggestions.....	3
1.4 Project Scope.....	3
1.5 References.....	3
2. Overall Description.....	4
2.1 Product Perspective.....	4
2.2 Product Features (Summary).....	4
2.3 User Classes and Characteristics.....	4
2.4 Operating Environment.....	4
2.5 Design and Implementation Constraints.....	5
2.6 Assumptions and Dependencies.....	5
3. System Features (Functional Requirements).....	6
3.1 Module: User Registration & Authentication.....	6
3.2 Module: Team & Match Explorer.....	6
3.3 Module: Prediction Engine.....	7
3.4 Module: Administrator Portal.....	7
3.5 Module: Notifications.....	8
4. External Interface Requirements.....	9
4.1 User Interfaces.....	9
4.2 Hardware Interfaces.....	9
4.3 Software Interfaces.....	9
4.4 Communication Interfaces.....	9
5. Use Case Summary.....	9
5.1 Primary Flow – Score Prediction (User).....	10
6. Non-Functional Requirements.....	11
6.1 Performance Requirements.....	11
6.2 Security Requirements.....	11
6.3 Usability.....	11
6.4 Reliability & Availability.....	11
6.5 Maintainability & Portability.....	11
6.6 Scalability.....	12
7. Software Testing Considerations.....	13
8. Proposed System Architecture.....	14
8.1 Suggested Technology Stack.....	14
9. Data Model Overview (for Database Planning).....	15
9.1 Core Entities.....	15
9.2 Key Relationships.....	15
10. Future Enhancements (Out of Current Scope).....	15
11. Glossary.....	16

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Cricket Score Predictor (CSP), a web-based application that predicts the likely final score and match outcome of a cricket match using historical team and player statistics combined with a machine-learning prediction engine. The document is intended to serve as the single source of truth for the development team, project guide, and evaluation committee regarding what the system will do, who will use it, and the constraints under which it must be built. It is written to be used for requirement analysis, UI/UX design, database and API planning, machine-learning model planning, software testing, and final project documentation.

1.2 Document Conventions

Functional requirements are uniquely identified using the format FR-XX, and are grouped by module. Each requirement is written using the keyword “shall” to indicate a mandatory capability of the system, consistent with IEEE 830 / ISO/IEC/IEEE 29148 conventions. Priority levels used throughout this document are High (must be implemented for a working system), Medium (important but can be simplified if time is short) and Low (optional / future enhancement).

1.3 Intended Audience and Reading Suggestions

- Students / Development Team – to understand what to build, module by module.
- Project Guide / Faculty Evaluator – to assess completeness, correctness and feasibility of scope.
- UI/UX Designer – to design screens based on Section 4 and Section 3 workflows.
- Database / Backend / ML Developer – to design schema, APIs and the prediction model based on Section 8 and Section 9.
- Software Tester – to derive test cases directly from the functional requirements in Section 3.

1.4 Project Scope

The Cricket Score Predictor will allow users to browse teams, players and past matches, select an upcoming or in-progress match, enter the current match state (overs bowled, wickets fallen, current run rate, venue, and playing squads), and receive a predicted final score, a projected score range and a win-probability estimate generated by a trained machine-learning model. Users can save/bookmark predictions, compare head-to-head team statistics, and review how accurate past predictions were once actual results are available. Administrators will manage team, player, venue and match records, oversee all generated predictions, trigger retraining of the prediction model with newly added match data, and generate basic operational and accuracy reports.

The scope is deliberately kept realistic for a college-level team project. The system predicts scores using historical statistical patterns and a trained regression/classification model — it is not a real-time broadcast, betting, or fantasy-sports platform. Live ball-by-ball ingestion from official broadcast feeds, real-money betting or fantasy contests, native mobile applications, and multi-sport support are explicitly out of scope for the current version and are listed as future enhancements in Section 10.

1.5 References

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Python (Flask/FastAPI), scikit-learn and MongoDB / MySQL official documentation.
- Publicly available historical cricket datasets (e.g. ball-by-ball IPL / ODI datasets) used for offline model training.

- Project description document / synopsis submitted by the student team.

2. Overall Description

2.1 Product Perspective

CSP is a new, self-contained, independent web application. It is not a modification of an existing scoring or broadcast system. It follows a standard three-tier web architecture (presentation, application/API, and data layer — detailed in Section 8) with a dedicated machine-learning prediction service, so that it can later be integrated with external systems such as live-score APIs as the project matures beyond the academic prototype stage.

2.2 Product Features (Summary)

- User registration, login and profile management.
- Team, player and match search with historical statistics.
- Score prediction based on current match state using a trained ML model.
- Win-probability and projected score-range display with confidence indicators.
- User dashboard for saved predictions and prediction history.
- Admin dashboard for managing teams, players, venues, matches and the ML model.
- Automated notifications when saved predictions are updated or matches conclude.
- Basic analytical reports (prediction accuracy, model performance, usage statistics).

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Visitor / Guest	General public exploring matches and predictions without an account.	Basic web/smartphone literacy; browses public predictions only.
Registered User	Cricket enthusiast who creates an account to save predictions.	Basic computer/smartphone literacy; primary end-user of the prediction flow.
Data Analyst (optional)	Assists in curating datasets and reviewing model accuracy.	Moderate technical familiarity with statistics and data files.
Administrator	Project owner / super-user managing data and the ML model.	Trusted internal user with full configuration and reporting access.

2.4 Operating Environment

- Client side: Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, tablet or smartphone with an active internet connection.
- Server side: Python (Flask/FastAPI) runtime for the prediction service and Node.js/Express for the core API, hosted on a cloud platform (e.g. Render, Railway, AWS/Azure free tier) or a local server for demonstration.
- Database: MongoDB Atlas (cloud) or local MongoDB / MySQL instance.
- Machine Learning: Model trained offline using Python (pandas, scikit-learn / XGBoost) and served via a lightweight REST prediction endpoint.

2.5 Design and Implementation Constraints

- Must be developed and demonstrated within a single college semester by a small student team (typically 2–4 members).
- Must primarily use free / open-source technologies and publicly available datasets to keep the project cost at zero.
- Must be responsive and usable on both desktop and mobile screen sizes.
- Live match data integration via a third-party API is treated as optional/future add-on due to cost and API-key verification overhead; the system shall support manual entry of match state as the default input method.
- The system assumes a single sport (cricket) and covers the T20, ODI and Test formats for the academic version.

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web forms and cricket terminology (overs, wickets, run rate).
- A historical match dataset is available and curated by the Administrator/Data Analyst before the prediction model is trained.
- Reliable internet connectivity is available at both client and server ends.
- Prediction accuracy depends on the quality and volume of historical data used to train the model, and is presented to users as an estimate, not a guarantee.

3. System Features (Functional Requirements)

This section lists the functional requirements of the system, grouped by module. Each requirement can be traced directly to a test case (see Section 7) and a screen (see Section 4).

3.1 Module: User Registration & Authentication

Description: Allows visitors to securely create accounts and log in, with role-based access.

ID	Requirement	Priority
FR-01	The system shall allow a new user to register using name, email, phone number and password.	High
FR-02	The system shall validate email format and enforce a minimum password strength on registration.	High
FR-03	The system shall authenticate registered users (User, Admin) via email and password login.	High
FR-04	The system shall issue a secure session/token (e.g. JWT) upon successful login.	High
FR-05	The system shall allow a user to reset a forgotten password via a one-time email link/OTP.	Medium
FR-06	The system shall restrict access to module features based on the logged-in user's role (User/Admin).	High
FR-07	The system shall allow a logged-in user to log out, invalidating the active session.	Medium

3.2 Module: Team & Match Explorer

Description: Core browsing experience for exploring teams, players and matches before requesting a prediction.

ID	Requirement	Priority
FR-08	The system shall allow a user to search/filter matches by team, tournament, venue or date.	High
FR-09	The system shall display each team's and player's profile including historical batting/bowling statistics.	High
FR-10	The system shall allow a user to select an upcoming, live or historical match to generate a prediction.	High
FR-11	The system shall prevent duplicate prediction requests for the same match state by reusing a cached result.	Medium
FR-12	The system shall allow a user to view a list of their previously generated and saved predictions.	High
FR-13	The system shall allow a user to bookmark/save a match prediction to their profile.	High
FR-14	The system shall allow a user to compare head-to-head statistics of two selected teams.	Medium
FR-15	The system shall allow a user to view and update their own profile and favourite teams.	Medium
FR-16	The system shall allow a user to view the prediction confidence score and	Medium

ID	Requirement	Priority
	the key factors behind it.	

3.3 Module: Prediction Engine

Description: Enables generation of score predictions and win probabilities using the trained ML model.

ID	Requirement	Priority
FR-17	The system shall allow input of current match state (overs bowled, wickets, current score, venue, teams) to generate a predicted final score.	High
FR-18	The system shall allow the match format (T20/ODI/Test) to be specified, selecting the appropriate prediction model.	High
FR-19	The system shall display the predicted score as a range with a confidence interval for the selected match.	High
FR-20	The system shall retrieve relevant historical team/player performance data before generating a prediction.	Medium
FR-21	The system shall support periodic retraining of the ML model as new match data is added, and mark predictions as validated once the actual result is available.	Medium
FR-22	The system shall allow a user to flag a prediction as inaccurate, notifying the Administrator for review.	Low

3.4 Module: Administrator Portal

Description: System configuration, oversight, dataset management and reporting.

ID	Requirement	Priority
FR-23	The system shall allow the Admin to add, edit, deactivate or remove team and player records.	High
FR-24	The system shall allow the Admin to create and manage tournaments, venues and match fixtures.	High
FR-25	The system shall allow the Admin to view and manage all registered user accounts.	Medium
FR-26	The system shall allow the Admin to view all generated predictions, with filters by date, team or match.	High
FR-27	The system shall allow the Admin to upload updated historical datasets and trigger model retraining.	High
FR-28	The system shall allow the Admin to generate reports on prediction accuracy, model performance and usage volume.	Medium
FR-29	The system shall allow the Admin to post system-wide announcements/notices visible to users.	Low

3.5 Module: Notifications

ID	Requirement	Priority
FR-30	The system shall send an email/in-app notification when a user's saved match prediction is generated or updated.	Medium
FR-31	The system shall send a notification comparing the predicted result with the actual result once a match concludes.	Medium
FR-32	The system shall send an automated reminder for a saved upcoming match prior to its scheduled start.	Low

4. External Interface Requirements

4.1 User Interfaces

- Landing / Home Page – project introduction, featured matches, login/register call-to-action.
- User Dashboard – search matches, request predictions, view/save predictions, favourite teams, profile.
- Prediction Screen – match-state input form, predicted score range, win probability, confidence factors.
- Admin Dashboard – manage teams/players/venues/matches, dataset upload, model retraining, reports and charts.
- All screens shall follow a consistent, clean, responsive layout (see Section 6.3) using a cricket-appropriate colour palette (pitch green with gold/stumps accents).

4.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard client devices (desktop/laptop/tablet/smartphone) with a working internet connection, and on any commodity server/cloud instance capable of running Node.js, Python and MongoDB/MySQL.

4.3 Software Interfaces

Component	Technology
Frontend	React.js (with React Router, Axios, and a component library such as Bootstrap/Tailwind/MUI)
Backend / API	Node.js with Express.js – RESTful JSON APIs
Prediction Service	Python (Flask/FastAPI) exposing the trained ML model via a REST endpoint
Machine Learning	pandas, scikit-learn / XGBoost for training and inference
Database	MongoDB (Mongoose ODM) or MySQL – stores users, teams, matches and predictions
Authentication	JWT (JSON Web Tokens) with bcrypt password hashing
Email Service	SMTP service (e.g. Nodemailer with Gmail/SendGrid) for notifications
Hosting (suggested)	Frontend – Vercel/Netlify; Backend/API – Render/Railway; Database – MongoDB Atlas

4.4 Communication Interfaces

- All client-server communication shall occur over HTTPS using RESTful JSON APIs.
- The core API shall communicate with the prediction service over an internal REST call.
- Outbound email notifications shall be sent via the SMTP protocol through the configured email service.
- (Future phase) Live match data may be integrated via a third-party cricket data API.

5. Use Case Summary

The table below summarizes the primary use cases per actor. Detailed step-by-step use-case descriptions (pre-conditions, main flow, alternate flow) can be expanded from this list during design if required by the evaluation format.

Actor	Use Case	Brief Description
User	Register / Login	Create an account or sign in to access prediction

Actor	Use Case	Brief Description
		features.
User	Search Team / Match	Find a match, team or player by name, venue or date.
User	Request Prediction	Enter current match state and generate a predicted score.
User	View / Save Prediction	Bookmark a prediction and review saved prediction history.
User	Compare Teams	View head-to-head statistics between two teams.
Admin	Manage Teams, Players & Venues	Add/edit/remove teams, players, venues and fixtures.
Admin	Manage Users	View or deactivate user accounts if required.
Admin	Upload Data & Retrain Model	Upload updated datasets and trigger ML model retraining.
Admin	View Reports	Analyse prediction accuracy, model performance and usage.

5.1 Primary Flow – Score Prediction (User)

1) User logs in → 2) Searches for an upcoming or in-progress match → 3) Enters the current match state (overs, wickets, score, venue, playing teams) → 4) System retrieves relevant historical team/player statistics → 5) Prediction Engine computes the predicted score range and win probability → 6) Result is displayed with a confidence indicator → 7) User may save the prediction, and the system logs it for future accuracy evaluation once the match concludes.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load any core page within 3 seconds under normal load on a standard broadband connection.	High
NFR-02	The system shall support at least 50–100 concurrent users without noticeable performance degradation (adequate for a college demo/pilot).	Medium
NFR-03	A score prediction shall be computed and returned within 3 seconds of submitting the match state.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	Passwords shall be stored using a one-way hashing algorithm (e.g. bcrypt); plain-text passwords shall never be stored.	High
NFR-05	All API endpoints that access personal data or predictions shall require a valid authentication token.	High
NFR-06	Role-based authorization shall ensure a standard User cannot access Admin-only endpoints.	High
NFR-07	All data in transit shall be encrypted using HTTPS/TLS.	High
NFR-08	User input shall be validated and sanitized on both client and server to prevent injection attacks.	High

6.3 Usability

- The interface shall be simple enough for a first-time user to generate a prediction in under 5 clicks/taps.
- The system shall be fully responsive across desktop, tablet and mobile screen sizes.
- Error and success messages shall be clear, specific, and shown close to the relevant form field/action.

6.4 Reliability & Availability

- The system shall target 99% uptime during the demonstration/evaluation period.
- Saved predictions and match data shall not be lost due to a single server restart (persisted in the database, not in memory).

6.5 Maintainability & Portability

- The codebase shall follow a modular MVC-style structure (routes/controllers/models on the backend, reusable components on the frontend) with the ML training pipeline kept as a separate, well-documented module to ease future maintenance.
- The system shall run on any OS capable of hosting Node.js, Python and MongoDB/MySQL (Windows, Linux, macOS), and shall be accessible from any modern browser without requiring a native app install.

6.6 Scalability

Although scoped for a single sport and a manually curated dataset in the academic version, the data model and API design shall avoid hard-coded single-tournament assumptions where reasonably possible, so that additional tournaments, formats and, eventually, live-data feeds can be added without a full redesign.

7. Software Testing Considerations

Each functional requirement in Section 3 shall map to at least one test case. The table below gives representative examples to guide the team's test plan; the full test case document can be maintained separately (e.g. in a spreadsheet) using this format.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-17	Request a prediction with valid match state	Predicted score range and win probability shown; result logged.
TC-02	FR-11	Submit an identical prediction request twice	System returns the cached result instead of recomputing.
TC-03	FR-13	Save/bookmark a prediction	Prediction appears in the user's saved-predictions list.
TC-04	FR-06	Standard user attempts to access Admin-only page	Access denied (403) / redirected to login.
TC-05	FR-02	Register with a weak password	Registration blocked with validation message.

Recommended testing types: Unit testing (backend controllers/validators and ML pre-processing functions using a framework such as Jest/Mocha and PyTest), Integration testing (API and prediction-service endpoints with Postman/Jest+Supertest), Model evaluation (accuracy/error metrics such as MAE and RMSE on a held-out test dataset), and Manual UI/UX testing across at least two browsers and one mobile viewport before final submission.

8. Proposed System Architecture

Given the team's familiarity and the project's realistic academic scope, a hybrid architecture combining a MERN-style web stack (MongoDB, Express.js, React.js, Node.js) with a dedicated Python-based Machine Learning prediction service is recommended. This keeps the web layer well documented and free to host on student-friendly free tiers, while isolating the ML training/inference logic so it can be developed and tested independently, mapping cleanly onto the module breakdown in Section 3.

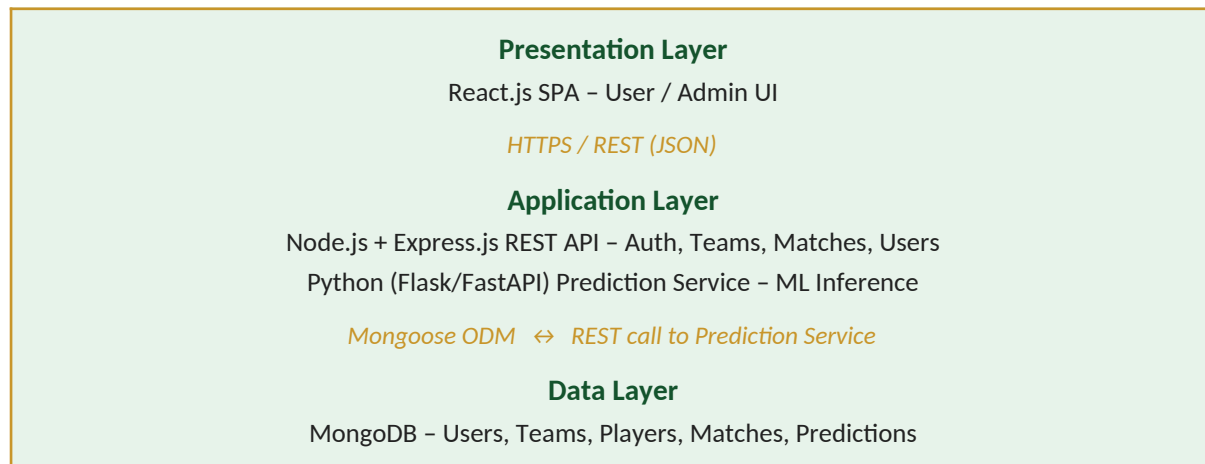


Figure 8.1: Three-tier architecture of the Cricket Score Predictor.

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, React Router, Axios, Bootstrap / Tailwind CSS
Backend (Core API)	Node.js, Express.js, JWT, bcrypt, Nodemailer
Prediction Service	Python, Flask/FastAPI, pandas, scikit-learn / XGBoost
Database	MongoDB with Mongoose (or MySQL)
Version Control	Git & GitHub
Testing	Postman (API testing), Jest (unit testing), PyTest (model testing)
Deployment (suggested)	Vercel/Netlify (frontend), Render/Railway (backend & prediction service), MongoDB Atlas (database)

9. Data Model Overview (for Database Planning)

The following core entities and relationships are proposed as a starting point for schema design. This is intentionally kept minimal and realistic for a student project; fields can be extended during implementation.

9.1 Core Entities

Entity	Key Attributes
User	_id, name, email, passwordHash, role (user/admin), phone, createdAt
Team	_id, name, country/franchise, logo, homeVenue
Player	_id, name, teamId, role (batter/bowler/all-rounder/keeper), battingAvg, strikeRate, bowlingAvg, economy
Venue	_id, name, city, avgFirstInningsScore
Match	_id, teamAId, teamBId, venueId, format (T20/ODI/Test), date, status (Upcoming/Live/Completed)
Prediction	_id, matchId, userId, matchStateSnapshot, predictedScoreRange, winProbability, confidenceScore, actualResult, createdAt
Notification	_id, userId, message, type, isRead, createdAt

9.2 Key Relationships

- A Team has many Players (one-to-many).
- A Match involves two Teams and one Venue (many-to-many via Match).
- A Match can have many Predictions; a User can generate many Predictions (many-to-many via Prediction).
- A Prediction is compared against the actual result once its Match is marked Completed (one-to-one, updated after the match).
- A User can receive many Notifications (one-to-many).

10. Future Enhancements (Out of Current Scope)

- Integration with a live cricket-data API for real-time ball-by-ball match state.
- Deep-learning based sequence models (e.g. LSTM) for improved in-match prediction accuracy.
- Native mobile application (Android/iOS).
- Multi-sport support (e.g. football, kabaddi score prediction).
- Player fitness, weather and pitch-condition factors as additional model inputs.
- Public prediction leaderboard and community accuracy challenges.

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – a web API architecture style
JWT	JSON Web Token – used for stateless authentication
ML	Machine Learning
MERN	MongoDB, Express.js, React.js, Node.js – a full JavaScript web stack
MAE / RMSE	Mean Absolute Error / Root Mean Squared Error – metrics used to evaluate prediction accuracy
CRUD	Create, Read, Update, Delete – basic data operations
OTP	One-Time Password
UI/UX	User Interface / User Experience

End of Document – Cricket Score Predictor SRS v1.0